```
#! /usr/bin/env python

"""Standalone tool to extract tables from logs.

The logs are viewed as a multidimensional map from a tuple of strings to a
number.  This is defined using pattern strings.

Over each of the fields, you can specify a filter constraining the output only
to records with specific value(s) of this field.

- Any field which has only one value in the final set (whether because of run
  parameters or focusing), is listed once in the caption of the table,
  reducing the dimensionality.

Then, you can average over the values of some fields, again reducing the
dimensionality.

"""

from __future__ import division

# I'm too lazy to implement something fully generic and universal...

import re
from sets import Set                    # Python 2.3 library

def parse_react(fname):
    """Return map from tuples to numbers from log of react run."""
    header = re.compile(r'==> react_([^-]*)-(hold)?_?([^_]*)_([^ ]*) <==')
    data = re.compile(r'\(([0-9.]*) < ([0-9.]*) \+- ([0-9.]*) < ([0-9.]*) x ([0-9.]*)\)')
    f = file(fname)
    d = {}
    for line in f:
        m = header.match(line)
        if m:
            h = m.groups('free')   # opposite of hold if not present
        m = data.match(line)
        if m:
            (d[h + ('min',)],
             d[h + ('avg',)],
             d[h + ('dev',)],
             d[h + ('max',)],
             d[h + ('cars',)]) = map(float, m.groups())
    f.close()
    return d

def parse_main(fname):
    """Return map from tuples to numbers from log of main run."""
    header = re.compile(r'==> ([^-]*)-([^_]*)_([^_]*)_([^_]*)_([^-]*)-([^ ]*) <==')
    data = re.compile(r'([^:]*): \(([0-9.]*) < ([0-9.]*) \+- ([0-9.]*) < ([0-9.]*) x ([0-9.]*)\)')
    f = file(fname)
    d = {}
    for line in f:
        m = header.match(line)
        if m:
            h = m.groups()
        m = data.match(line)
        if m:
            roads = m.groups()[0]
            (d[h + ('min', roads)],
             d[h + ('avg', roads)],
             d[h + ('dev', roads)],
             d[h + ('max', roads)],
             d[h + ('cars', roads)]) = map(float, m.groups()[1:])
    f.close()
    return d

def limit(d, *sets):
    """For each true sets[i], limit keys from d to be in sets[i]."""
    def ok(key, sets):
```

```
        for field, set in zip(key, sets):
            if set and field not in set:
                return 0
        return 1
    return dict([(k, v) for (k, v) in d.items() if ok(k, sets)])

def average(d, *sets):
    """Average over field i for each true sets[i], set field from sets[i]."""
    d2 = {}
    for key, val in d.items():
        key2 = tuple([set or field for field, set in map(None, key, sets)])
        vals = d2.setdefault(key2, [])
        vals.append(val)
    def avg(lst):
        from operator import add
        return reduce(add, lst) / len(lst)
    return dict([(k, avg(v)) for (k, v) in d2.items()])

def reorder(d, order):
    """Put key[order[i]] at place i."""
    return dict([(tuple(map(k.__getitem__, order)), v)
                 for (k, v) in d.items()])

def writetable(d, outfname):
    """Write out LaTeX fragment with (possibly nested) table."""
    # Find fields with single value common to all records
    sets = [Set() for field in d.iterkeys().next()]
    for key in d:
        for field, set in zip(key, sets):
            set.add(field)
    commons = [len(set) == 1 and iter(set).next() for set in sets]
    def lowdim(key):
        """Reduce dimensionality"""
        return tuple([field for field, common in zip(key, commons)
                      if not common])
    d = dict([(lowdim(k), v) for k, v in d.items()])
    sets = lowdim(sets)
    print commons
    print sets
    # Write table
    def emph(s):
        return '\\emph{' + str(s) + '}'
    def write(d, sets, focus = []):
        """Recursively produce table"""
        d = limit(d, *[[field] for field in focus])
        dim = len(sets) - len(focus)
        if dim == 0:
            try:
                return '%.3f' % (d.itervalues().next(),)
            except StopIteration:
                return '---'
        elif dim % 2 == 1:
            vkeys = list(sets[len(focus)])
            vkeys.sort()
            return ('\\begin{tabular}{l||l}\n' +
                    ' \\\\\n\\hline\n'.join([emph(vkey) + ' & ' +
                                            write(d, sets, focus + [vkey])
                                            for vkey in vkeys]) +
                    ' \\\\\n\\end{tabular}\n')
        else:
            hkeys = list(sets[len(focus)])
            vkeys = list(sets[len(focus) + 1])
            hkeys.sort()
            vkeys.sort()
            return ('\\raisebox{0pt}[1.5\\depth][1.1\\height]{\n' +
                    '\\begin{tabular}[c]{l||' + '|'.join('l'*len(hkeys)) + '}\n' +
                    '& ' + ' & '.join(map(emph, hkeys)) +
                    ' \\\\\n\\hline\n\hline\n' +
                    ' \\\\\n\\hline\n'.join(
                        [emph(vkey) + ' & ' +
```

```
                        ' & '.join([write(d, sets, focus + [hkey, vkey])
                                    for hkey in hkeys])
                        for vkey in vkeys]) +
                    ' \\\\\n\\end{tabular}\n' +
                    '}\n')
    f = file(outfname, 'w')
    print >>f, write(d, sets)
    f.close()

def do_react():
    """Generate the table for the react part."""
    # A bug was patched by separate partial re-run
    d = parse_react('results/react.log')
    d.update(parse_react('results3/react.log'))
    # from: load, hold, policy, size, result
    #   to: size, load, hold, result, policy
    d = reorder(d, [3, 0, 1, 4, 2])
    d = limit(d, ['10x10', '15x15'], None, None, ['avg', 'dev'])
    d = average(d, 1)
    writetable(d, 'react.tex')

def do_main():
    """Generate the tables for the main part."""
    d = parse_main('results4/main.log')
    # orig order: load, hold, flex, wisdom, lru, plan, result, roads

    # Compare 3 roads with 4 roads
    #  new order: plan, load, hold, flex, wisdom, lru, result, roads
    d34 = reorder(d, [5, 0, 1, 2, 3, 4, 6, 7])
    d34 = limit(d34, None, None, ['hold2'], ['fix'],
                None, ['lru'], ['avg', 'dev'], ['3', '4'])
    d34 = average(d34, 1)                # over plans
    writetable(d34, 'main34.tex')
    # From here limit to only '4'

    # Compare the 3 plans
    #  new order: load, hold, flex, wisdom, lru, result, plan, roads
    dplans = reorder(d, [0, 1, 2, 3, 4, 6, 5, 7])
    dplans4 = limit(dplans, None, ['hold2'], ['fix'], None, ['lru'],
                    ['avg', 'dev'], None, ['4'])
    writetable(dplans4, 'mainplans4.tex')
    dplans3 = limit(dplans, None, ['hold2'], ['fix'], None, ['lru'],
                    ['avg', 'dev'], None, ['3'])
    writetable(dplans3, 'mainplans3.tex')

    # Compare flex with fix, as function of hold
    #  new order: plan, load, hold, flex, lru, result, wisdom, roads
    dflex = reorder(d, [5, 0, 1, 2, 4, 6, 3, 7])
    dflex = limit(dflex, None, ['0.20'], None, None, ['lru'],
                  ['avg', 'dev'], None, ['4'])
    dflex = average(dflex, 1)            # over plans
    writetable(dflex, 'mainflex.tex')

    # Compare wisdoms, as function of load
    #  new order: plan, flex, hold, load, lru, result, wisdom, roads
    dwisdoms = reorder(d, [5, 2, 1, 0, 4, 6, 3, 7])
    dwisdoms = limit(dwisdoms, None, ['fix'], None, None, ['lru'],
                     ['avg', 'dev'], None, ['4'])
    dwisdoms = average(dwisdoms, 1)          # over plans
    writetable(dwisdoms, 'mainwisdoms.tex')


if __name__ == '__main__':
    do_react()
    do_main()
```